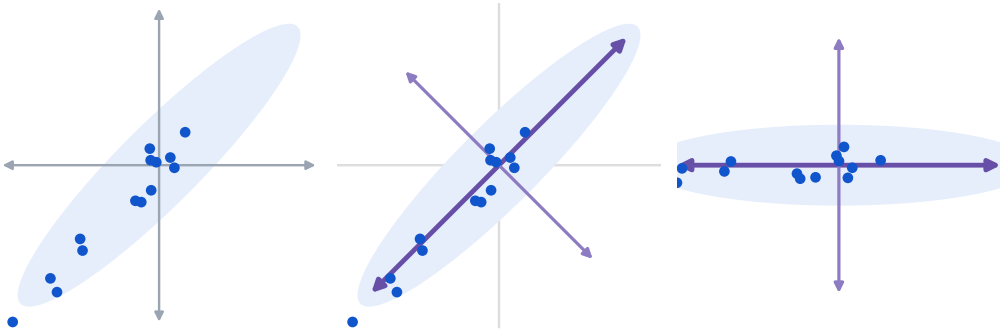


UNIT 02 · CHAPTER 8

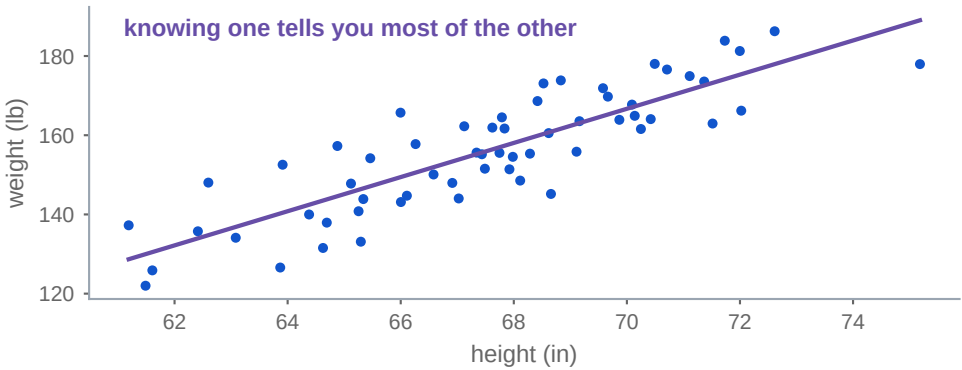
Principal Component Analysis

CSE 30124 · Introduction to Artificial Intelligence
William Theisen · University of Notre Dame

Too many features

Say we want to predict a student's GPA. We collect everything we can think of: **height, weight, hours of sleep, credit load, study time** and **assignment completion rate**.

Do you think all six features are equally useful? Probably not! Some tell us almost nothing about GPA. Others **tell us the same thing twice:**



Height and weight rise and fall together, so keeping both gives our model two columns but only about one column's worth of information.

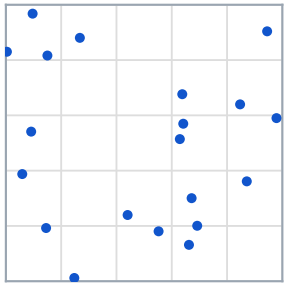
Every chapter so far has been about *using* features. This one is about **getting rid of them** without losing what they tell us.

The curse of dimensionality

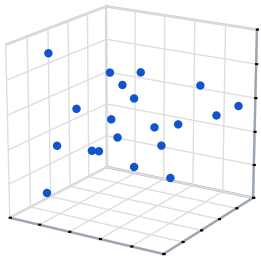
Why not just keep every feature? As we add more **dimensions** to our dataset, our data becomes more **spread out**. Here are the same 20 points with one, two and three features:



1 feature: 5 cells



2 features: 25 cells



3 features: 125 cells

With one feature, the points crowd into 5 cells. With three, the same 20 points rattle around in 125 cells, and most cells are empty. The space grows **exponentially**, but our data doesn't.

This is the **curse of dimensionality**: in high dimensions every point looks far away from every other point, so models that rely on "nearby" points (like kNN) have very little to go on.

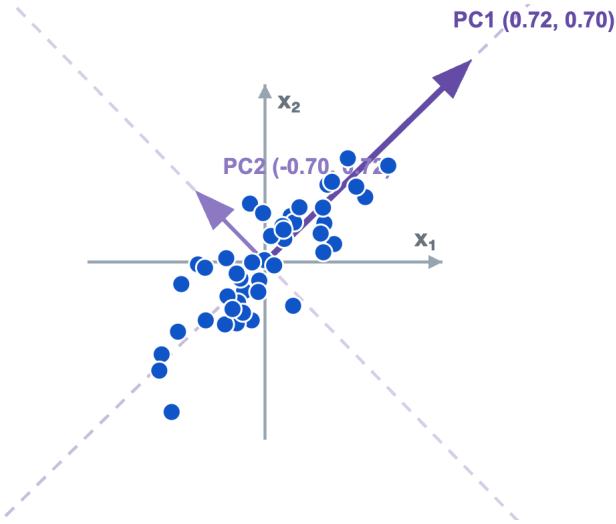
Quick check: we split every feature into 5 cells. How many cells are there with 6 features? With our 20 points, at most how many of them are occupied?

A new set of axes

Principal Component Analysis (PCA) is an **unsupervised learning** algorithm. It never looks at labels, only at the features themselves.

It finds us a **new set of axes** for our data, chosen so that:

- **Most of the variance** in the dataset lies along the first few axes.
- The axes are **orthogonal** to each other, so the new features are **uncorrelated**.



Nothing about the data changes. PCA is just a **rotation**. We turn our heads until the data is lined up with the axes. Then, if the last few axes barely matter, **we can drop them.**

The road ahead

PCA is five steps. We'll build up the two ideas they rest on, then do every step by hand:

1. **Standardize the data** — page 10
   Zero mean and unit variance for every feature.
2. **Covariance matrix** — page 12
   How much every pair of features moves together.
3. **Eigenvalues and eigenvectors** — pp. 13–16
   The new axes, and the variance along each.
4. **Sort them** — page 17
   Largest variance first.
5. **Choose how many to keep** — pp. 18–26
   Drop the axes that barely matter.

Notation

x_i	one example (a row); n examples, d features
μ, σ	a feature's average and standard deviation
C	the covariance matrix (d by d)
v	a direction (a row of length 1): a new axis
λ	the variance along that direction
$z = x\, v^T$	a point's position along the new axis

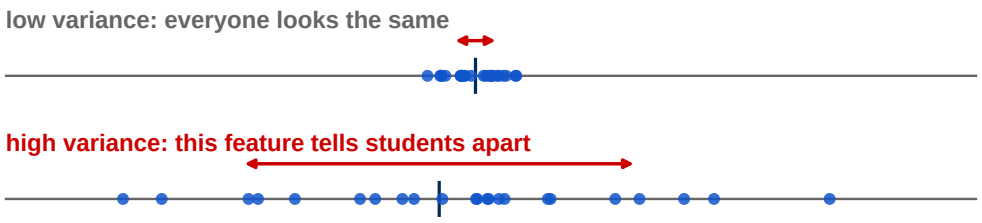
Variance

A dataset is only interesting if its samples are different from each other. If every student had the same study time, study time couldn't explain anything!

We measure how spread out a feature is with its **variance**. It's the average squared distance from the feature's average μ.

[8.1] variance of one feature

$$\mathrm{Var}(x) = \frac{1}{n-1}\sum_{i}(x_i - \mu)^2$$



Typically, the **higher the variance** of a feature, the **more "information"** it can give us about the data.

Why n − 1? We estimated μ from the same data, which uses up one data point's worth of freedom. Its square root, σ, is the standard deviation.

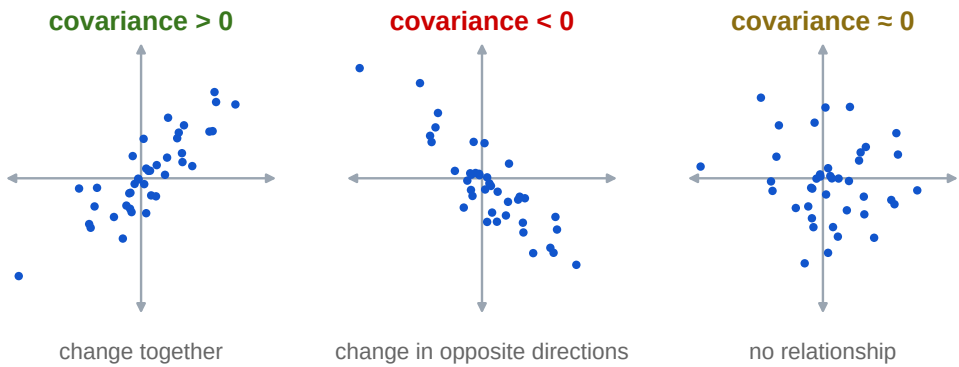
Covariance

Variance looks at one feature. Features in our dataset often have some relationship *to each other*, like height and weight. We measure that with the **covariance** between two features a and b:

[8.2] covariance of two features

$$\mathrm{Cov}(a, b) = \frac{1}{n-1} \sum_i (a_i - \mu_a)(b_i - \mu_b)$$

Each term is positive when both features are on the same side of their averages, and negative when they're on opposite sides:



The more two features overlap **(high covariance)**, the more overlapping information they carry.

Quick check: what is the covariance of a feature with itself?

The covariance matrix

With d features there's a covariance for every pair. We collect them all in one d by d table, the **covariance matrix** C. For two features a and b:

$$C = \begin{bmatrix} \mathrm{Var}(a) & \mathrm{Cov}(a,b) \\ \mathrm{Cov}(a,b) & \mathrm{Var}(b) \end{bmatrix}$$

- The **diagonal** holds each feature's variance.
- Everything **off the diagonal** is a covariance, and the matrix is symmetric.

If each feature already has its average subtracted, and we stack our examples as the rows of a matrix X, one line computes the whole thing:

covariance matrix of centered data

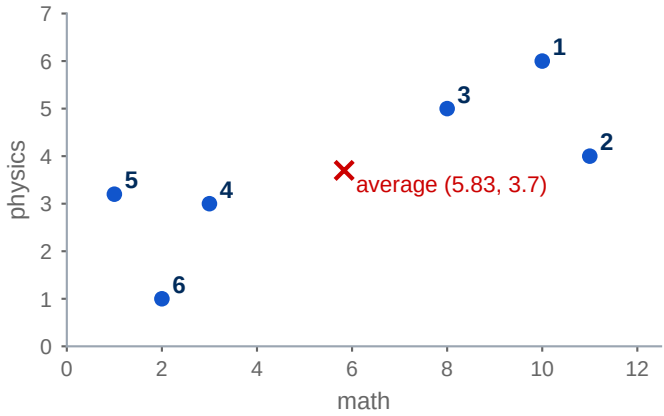
$$[8.3] \qquad C = \frac{1}{n-1} X^{\mathsf{T}} X$$

PCA's goal in one sentence: find new axes where this matrix has zeros everywhere off the diagonal.

Calculating PCA

Say we have the following **dataset** of six students' scores in Math and Physics:

Student	Math	Physics
1	10.0	6.0
2	11.0	4.0
3	8.0	5.0
4	3.0	3.0
5	1.0	3.2
6	2.0	1.0



Students who do well in one tend to do well in the other. Two features, but it looks like there's really **one story: "how strong is this student?"** Let's see if PCA finds it.

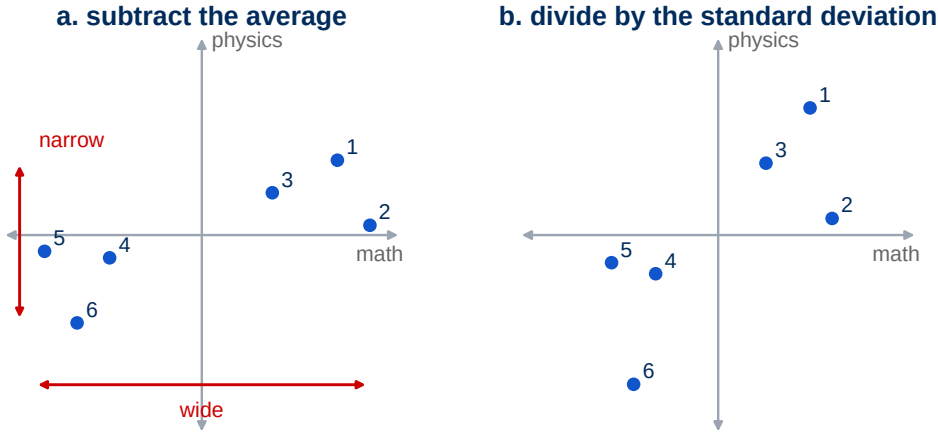
1. Standardize the data

a. PCA first calculates the **average** of each feature and subtracts it, so the data is **centered around the origin**. Our averages are 5.83 for math and 3.70 for physics.

b. We then divide each feature by its **standard deviation**, so both have a variance of exactly 1. Ours are 4.36 for math and 1.74 for physics.

[8.4] standardize one value

$$x \leftarrow \frac{x - \mu}{\sigma}$$



Why bother with b? PCA chases variance. Math scores are spread over 10 points and physics over 5, so without scaling PCA would mostly just rediscover the math axis. When every feature is already in the same units, step b is optional.

Our standardized data

Student 1's math score becomes `(10 − 5.83)/4.36 = 0.96`: almost one standard deviation above average. Here's everyone:

Student	Math	Physics	Product
1	0.96	1.32	1.27
2	1.19	0.17	0.20
3	0.50	0.75	0.37
4	−0.65	−0.40	0.26
5	−1.11	−0.29	0.32
6	−0.88	−1.55	1.37

Both columns now average 0 with variance 1. The last column multiplies each student's two scores. We'll need it on the next page.

Quick check: every product in the table is positive. What does that tell you about the covariance before we even compute it?

2. The covariance matrix

After standardizing, both variances are 1, so the diagonal is done. For the covariance, both averages are now 0, so [8.2] is just the sum of the products divided by n − 1:

$$\frac{1.27 + 0.20 + 0.37 + 0.26 + 0.32 + 1.37}{5} = 0.76$$

↓ fill in the matrix

our covariance matrix

$$[8.5] \quad C = \begin{bmatrix} 1 & 0.76 \\ 0.76 & 1 \end{bmatrix}$$

From the **covariance matrix** we can read the **relationships** between our features:

- **Covariance > 0:** features change together.
- **Covariance < 0:** features change opposite of each other.
- **Covariance ≈ 0:** features mostly don't have a relationship.

Ours is 0.76: strongly positive, just like the picture.

Correlation: the covariance of standardized features always lands between −1 and +1. It's better known as the correlation.

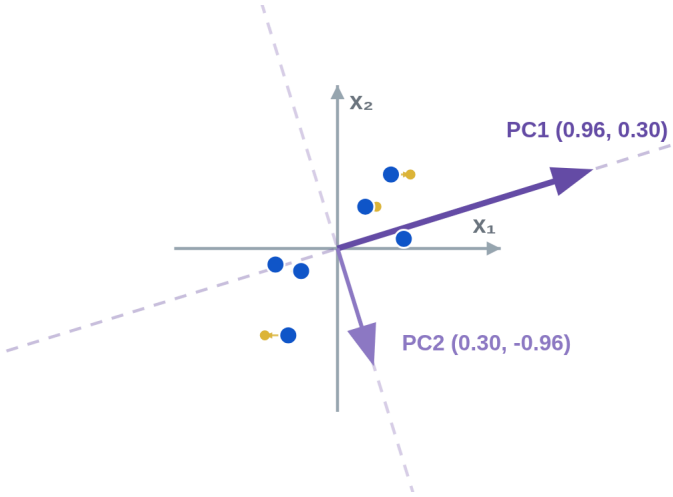
What are we looking for?

A **direction** is a row v of length 1. A point's position along it is a dot product, just like a score in the last few chapters:

[8.6] position of x along direction v

$$z = x\,v^{T}$$

Measure all six students along v and we get six numbers, with some variance. Turn v and the variance changes:



The first new axis is the direction with the **most variance**. Conveniently, we don't have to re-measure for every direction. The covariance matrix already knows the answer:

[8.7] variance along direction v

$$\mathrm{Var}(z) = v\,C\,v^{T}$$

Why eigenvectors?

We want the v that makes [8.7] as large as possible, but only among directions of length 1. That's an optimization with a constraint, so we use a **Lagrange multiplier**, exactly like Chapter 6:

variance, with a penalty for leaving length 1

$$L(v, \lambda) = v C v^T - \lambda (v v^T - 1)$$

$$\downarrow \text{ set } \frac{\partial L}{\partial v} = 0$$

[8.8] the eigenvector equation

$$C v^T = \lambda v^T$$

Read that last line slowly. Multiplying v by C doesn't turn it, it only stretches it by λ. Directions like that are called **eigenvectors** of C, and their stretch factors λ are **eigenvalues**.

Even better, plug [8.8] back into [8.7]:

$$\text{Var}(z) = v (\lambda v^T) = \lambda$$

- The **eigenvectors** are the directions our new axes point.
- The **eigenvalues** are the variance along each of them.

3. Eigenvalues

Move everything in [8.8] to one side: $(C - \lambda I)v^T = 0$. A nonzero v can only be squashed to zero if the matrix $C - \lambda I$ has a determinant of 0. That gives us an equation with only λ in it:

[8.9] the characteristic equation

$$\det(C - \lambda I) = 0$$

↓ our C, with λ subtracted on the diagonal

$$\det\begin{bmatrix} 1-\lambda & 0.76 \\ 0.76 & 1-\lambda \end{bmatrix} = 0$$

↓ 2 × 2 determinant: ad − bc

$$(1-\lambda)^2 - 0.76^2 = 0$$

↓ square root of both sides

$$1 - \lambda = \pm 0.76$$

Two answers, one for each new axis:

[8.10] $\lambda_1 = 1.76 \qquad \lambda_2 = 0.24$

Check: they add up to 2, the total variance we started with (1 for math plus 1 for physics). Rotating the axes moves variance around; it never creates or destroys any.

3. Eigenvectors

Now we find the direction that goes with each eigenvalue. Put $\lambda_1 = 1.76$ back into $(C - \lambda I)v^T = 0$, writing $v_1 = [a, b]$:

$$\begin{bmatrix} -0.76 & 0.76 \\ 0.76 & -0.76 \end{bmatrix}\begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

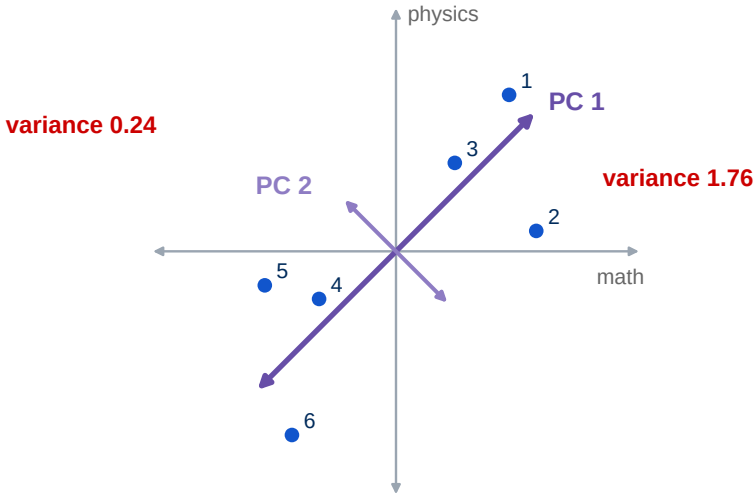
↓ both rows say the same thing

$$a = b$$

↓ scale to length 1

$$[8.11]\quad v_1 = \frac{1}{\sqrt{2}}[1,\ 1] \approx [0.71,\ 0.71]$$

Doing the same for $\lambda_2 = 0.24$ gives $a = -b$, so $v_2 = [-0.71, 0.71]$.



Orthogonal: $v_1\ v_2^T = -0.5 + 0.5 = 0$. PC 2 is the direction of most variance among those perpendicular to PC 1. A symmetric matrix like C always has perpendicular eigenvectors.

4. Sort them

We sort the eigenvalues from largest to smallest, and their eigenvectors along with them. The sorted directions are the **principal components**:

	Direction	Variance λ	Share
PC 1	`[0.71, 0.71]`	1.76	**88%**
PC 2	`[-0.71, 0.71]`	0.24	12%

A component's share is its eigenvalue divided by the total. It's called the **explained variance ratio**:

[8.12] share of the variance on component k

$$\frac{\lambda_k}{\lambda_1 + \lambda_2 + \dots + \lambda_d}$$

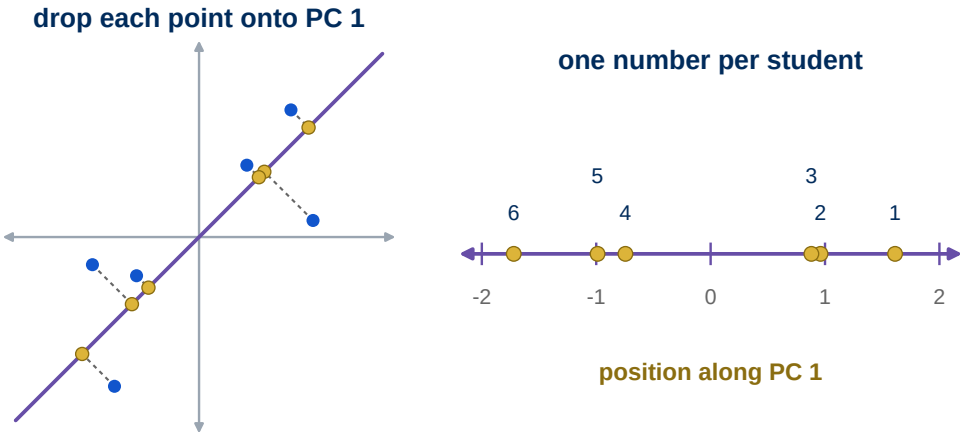
Can we name the new axes? PC 1 adds the two (standardized) scores: it's **overall strength**, the "one story" we guessed on page 9. PC 2 is physics minus math: **which subject a student leans toward.**

Careful: with real data the components are rarely this easy to name. Each one is a mixture of every original feature.

5. Choose and project

PC 1 holds 88% of the variance, so let's keep only that one. Each student's new feature is their position along it. Using `[8.6]` for student 1:

$$z = x\, v_1^{\top} = 0.707(0.96) + 0.707(1.32) = 1.61$$



Student	1	2	3	4	5	6
Position on PC 1	1.61	0.96	0.88	−0.74	−0.99	−1.72

We went from two features to one, and **kept 88% of what made these students different.** That's **dimensionality reduction**.

Quick check: a new student is exactly average in math and one standard deviation above average in physics. Where do they land on PC 1?

Did it work?

On page 8 we said PCA's goal was a covariance matrix with zeros off the diagonal. Let's check. Here is every student measured along *both* new axes:

Student	PC 1	PC 2	Product
1	1.61	0.26	0.42
2	0.96	−0.72	−0.69
3	0.88	0.18	0.16
4	−0.74	0.18	−0.13
5	−0.99	0.58	−0.57
6	−1.72	−0.48	0.82

This time the products cancel: they add up to zero. Each column's variance is its eigenvalue, so in the new axes:

covariance matrix in the new axes

[8.13] $$C_{\text{new}} = \begin{bmatrix} 1.76 & 0 \\ 0 & 0.24 \end{bmatrix}$$

The new features are **uncorrelated**. Knowing a student's overall strength tells us nothing about which subject they lean toward. All the overlap between math and physics has been rotated into PC 1.

Quick check: compare the Product column here with the one on page 11. What changed?

Going back

We kept one number per student. Can we get their two scores back? Almost: walk z steps along the direction we kept.

`[8.14]` rebuild a point from its position along v

$$\hat{x} = z\,v$$

For student 1, $z = 1.61$:

$$\hat{x} = 1.61\,[0.707,\ 0.707] = [1.14,\ 1.14]$$

Their real standardized scores were `[0.96, 1.32]`. We miss by `[−0.18, 0.18]`, which is exactly their part along PC 2, the axis we dropped.

- **What we lose** is the variance on the dropped components. Here that's $\lambda_2 = 0.24$, the missing 12%.
- **Back to real units:** undo step 1 by multiplying by σ and adding μ. Student 1 comes back as about 10.8 in math and 5.7 in physics. They really scored 10 and 6.

Keep k components and there's one such term for each. Keep all of them and we get the point back exactly.

$$\hat{x} = z_1 v_1 + z_2 v_2 + \cdots + z_k v_k$$

That won't scale

With two features the characteristic equation [8.9] was a quadratic. With d features it's a polynomial of degree d, and for d above 4 there isn't even a formula for its roots.

Computers don't solve it that way. Finding the roots of a big polynomial is **unstable and expensive**. Tiny rounding errors in C turn into large errors in λ. Even building C multiplies our data by itself, which makes those rounding errors worse.

We instead use something called the **Singular Value Decomposition** (SVD), which works on the data directly:

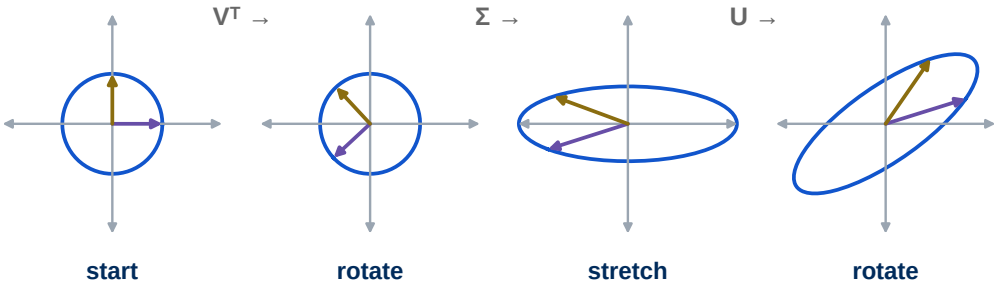
[8.15] singular value decomposition of any matrix

$$X = U \Sigma V^T$$

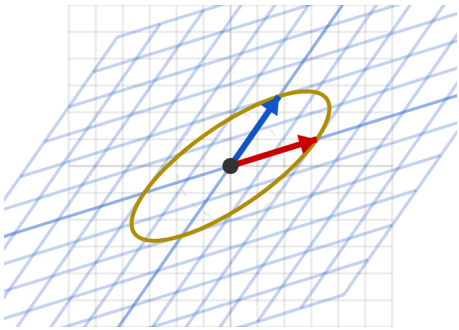
Remember, PCA is just trying to find a **new set of axes for the data**, and a change of axes is a **linear transformation**. The SVD says something remarkable about *every* linear transformation.

Rotate, stretch, rotate

Any **linear transformation** can be described as a rotation, then a stretch along the axes, then another rotation. Watch what a matrix does to a circle, one piece at a time:



- V^T **rotates.** Its rows are perpendicular directions of length 1.
- Σ **stretches.** It's zero except on its diagonal, which holds the **singular values** $\sigma_1 \geq \sigma_2 \geq \ldots$, already sorted.
- U **rotates** again.



Perpendicular directions, sorted by how much they stretch. That should sound a lot like principal components!

The SVD hands us PCA

Take the covariance matrix [8.3] and replace X with its SVD. Because U is a rotation, $U^T U = I$ and it cancels:

$$C = \frac{1}{n-1} X^T X = \frac{1}{n-1} V \Sigma U^T U \Sigma V^T$$

$$\downarrow U^T U = I$$

$$[8.16] \quad C = V \frac{\Sigma^2}{n-1} V^T$$

That's C written as rotate, stretch, rotate back: exactly what eigenvectors and eigenvalues describe. So:

- The rows of V^T are the **eigenvectors**: our principal components.
- Σ holds the **eigenvalues**, after squaring and dividing by $n - 1$:

$$[8.17] \quad \lambda_k = \frac{\sigma_k^2}{n-1}$$

Steps 2, 3 and 4 all come out of one decomposition, **and we never build C at all.**

PCA in numpy

Here's our whole worked example. Standardize, then one call to the SVD:

```python
import numpy as np

# math and physics scores for six students
X = np.array([[10, 6], [11, 4], [8, 5],
              [3, 3], [1, 3.2], [2, 1]])

# step 1: standardize each column
Z = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)

# steps 2 to 4 in one call
U, S, Vt = np.linalg.svd(Z, full_matrices=False)
print(Vt.round(3))                      # directions
print(S.round(2))                       # singular values
print((S**2 / (len(X) - 1)).round(3))   # variances

# step 5: keep only PC 1
z = Z @ Vt[0]
print(z.round(2))
```

OUTPUT

```
[[-0.707 -0.707]
 [-0.707  0.707]]
[2.97 1.1 ]
[1.758 0.242]
[-1.61 -0.96 -0.88  0.74  0.99  1.72]
```

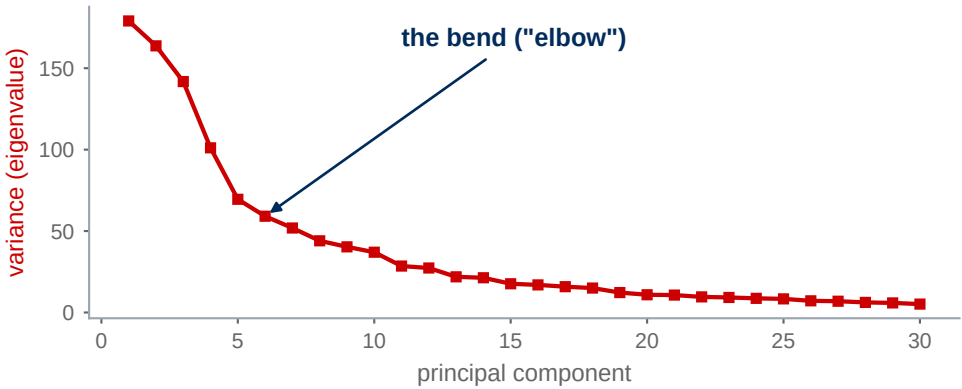
The same eigenvalues, 1.76 and 0.24, with no determinant in sight. The singular values are 2.97 and 1.10, and $2.97^2/5 = 1.76$, just as `[8.17]` says.

Flipped signs: numpy's PC 1 points down and to the left, so every position has the opposite sign from ours. It's the same axis: a "rotation" is allowed to include a mirror flip. Only the order of the students along it matters.

Scree plots

Two features were enough to learn the steps. Let's try a real dataset: 1,797 images of **handwritten digits**, each 8 by 8 pixels. That's **64 features** per image, and so 64 principal components. Every pixel is on the same brightness scale, so here we only center the data and skip step 1b.

It's useful to plot the variance of each component, in order. This is called a **scree plot**:



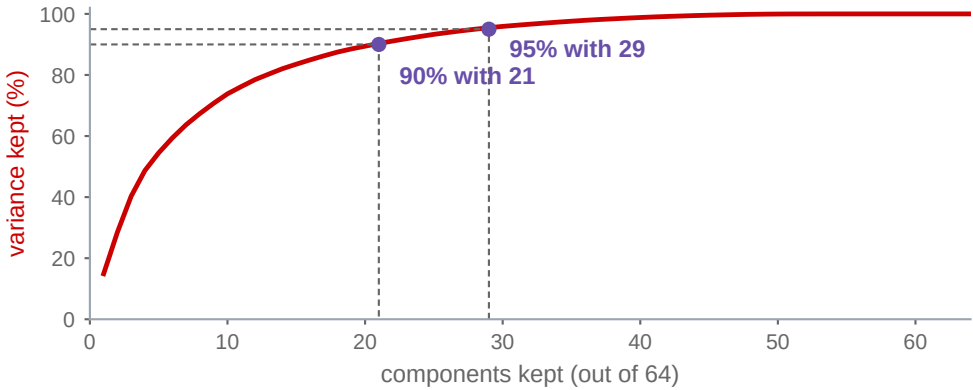
A scree plot typically drops quickly and then flattens out. The bend is called the **elbow**. Components before it carry a lot of variance each; after it, each one adds very little.

The new axes **"front-load" the variance**. That's the whole point: we can usually keep a handful of components and **not seriously impact our results.**

Scree is the loose rock piled at the bottom of a cliff, which is what the flat tail of the plot looks like.

How many should we keep?

The elbow is a judgment call. A more precise rule is to add up the explained variance ratios [8.12] as we go, and stop once we've kept enough of it:



For our digits, **21 components keep 90% of the variance** and 29 keep 95%. We can throw away more than half of our features and lose a twentieth of the variance!

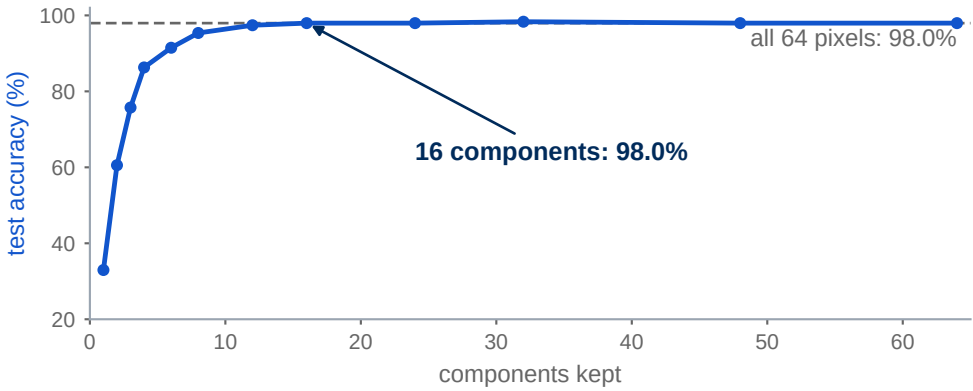
Here's one digit rebuilt from only its first few components, using [8.14]:



No right answer: 90% and 95% are common choices, not laws. The number of components is a hyperparameter, so the honest way to pick it is on validation data.

Does the model mind?

Variance kept is nice, but what we care about is whether our model still works. Here's a **kNN** classifier trained on the digits after keeping only the first few components:

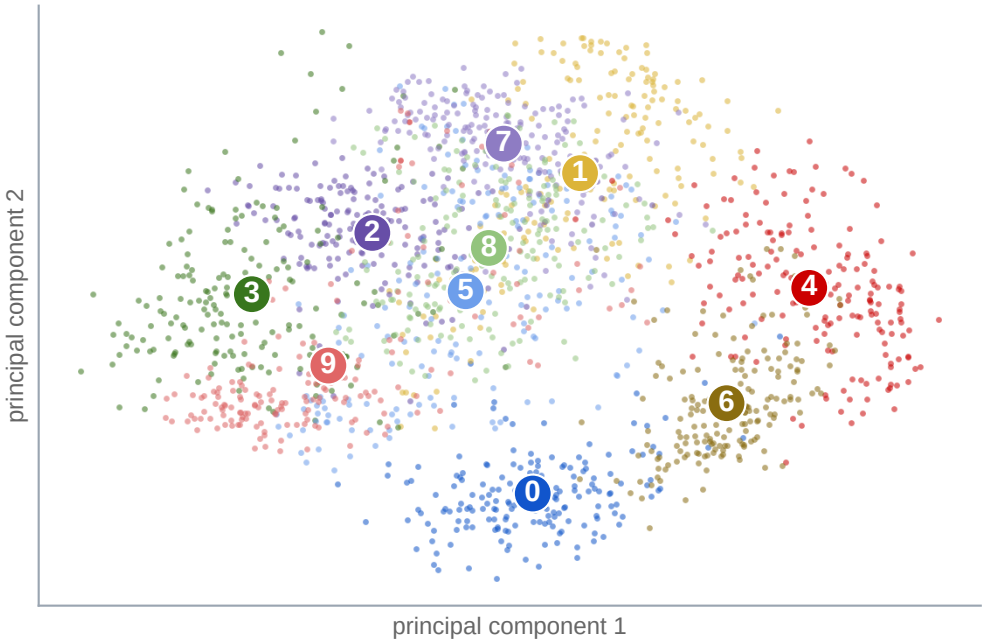


With just **16 of the 64 components** we match the accuracy of using every pixel. Remember the curse of dimensionality from page 3? Fewer dimensions means:

- **Faster** training and prediction, and less to store.
- **Denser data**, so "nearby" means something again.
- **Less noise.** The components we drop are often mostly noise, so some models even improve.

Enabling visualizations

Another benefit of **PCA** is that it lets you **see a dataset**. We can't plot 64 dimensions, but we can plot the first two principal components:



Each dot is one image, colored by which digit it is. PCA never saw those labels, yet many of the digits form their own clumps. And this is with **less than 30% of the variance!**

Read with care: digits that overlap here (like 5 and 8) may be well separated along components we aren't showing.

PCA in Python

In practice we use scikit-learn's `PCA`, which centers the data and runs the SVD for us:

```python
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_digits(return_X_y=True)   # 64 pixels per image
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

# learn the new axes from the training data only
pca = PCA(n_components=16).fit(X_tr)
Z_tr, Z_te = pca.transform(X_tr), pca.transform(X_te)
print(Z_tr.shape)
print(round(pca.explained_variance_ratio_.sum(), 3))

knn = KNeighborsClassifier(5).fit(Z_tr, y_tr)
print(round(knn.score(Z_te, y_te), 3))
```

OUTPUT

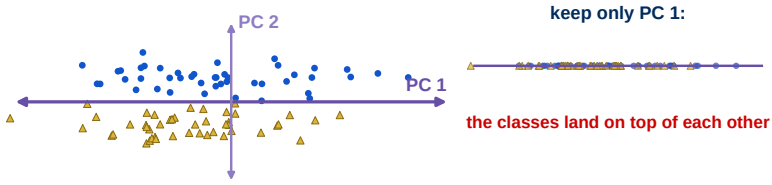
```
(1257, 16)
0.851
0.98
```

16 features instead of 64, 85% of the variance, and 98% test accuracy. `PCA(n_components=0.95)` instead keeps however many components reach 95%.

Scaling: `PCA` centers but does not scale. Pixels share one unit, so we skipped it here. When features have different units, put a `StandardScaler` in front.

Where PCA misleads

PCA is unsupervised: it keeps the directions with the most variance, and **the most variance isn't always the most useful.**



Here the classes differ along PC 2, the direction PCA would throw away first. A few more things to watch for:

- **It's linear.** PCA can rotate, but it can't unbend. Curved structure needs something like the kernels of Chapter 6.
- **It's scale sensitive.** Measure a feature in different units and you get different components. Standardize first.
- **It's harder to explain.** "Component 3" is a mixture of every feature, not something you can point to.
- **It's swayed by outliers.** One extreme point can pull a whole component toward itself.
- **It can leak.** Fit PCA on the training data only, then apply those same axes to validation and test data.

Cheat sheet

The whole algorithm

1. **Standardize:** subtract each feature's average, divide by its standard deviation.
2. **Covariance matrix** C.
3. **Eigenvectors and eigenvalues** of C.
4. **Sort** by eigenvalue, largest first.
5. **Keep** the first k and project.

[8.3] covariance matrix

$$C = \frac{1}{n-1} X^T X$$

[8.8] directions and their variance

$$C v^T = \lambda v^T$$

[8.6] new feature, and going back

$$z = x v^T \qquad \hat{x} = z v$$

[8.12] explained variance ratio

$$\frac{\lambda_k}{\lambda_1 + \lambda_2 + \cdots + \lambda_d}$$

[8.17] from the SVD

$$X = U \Sigma V^T, \qquad \lambda_k = \frac{\sigma_k^2}{n-1}$$

Remember

- **Eigenvectors** are directions; **eigenvalues** are variance.
- The components are **orthogonal**, so the new features are uncorrelated.
- Rotating never changes the **total** variance.
- PCA is **unsupervised** and **linear**. Scale first; fit on training data only.

Practice problems

Try these before peeking at the answers!

1. **Variance by hand.** A feature has the values 2, 4 and 6. What are its average, variance and standard deviation?
2. **Reading a matrix.** Two features have variances 4 and 9, and covariance −3. Write their covariance matrix. Do they move together or opposite?
3. **Two components.** Two standardized features have a covariance of 0.5. Find both eigenvalues and the share of the variance on PC 1.
4. **Projecting.** A standardized point is $\mathbf{x}$ = `[2, 1]` and PC 1 is $\mathbf{v}$ = `[0.6, 0.8]`. Is $\mathbf{v}$ a valid direction? Where does $\mathbf{x}$ land on it?
5. **How many?** A dataset's eigenvalues are 5, 3, 1.5 and 0.5. How much variance do the first two components keep? How many do we need for 95%?
6. **From the SVD.** A centered dataset with 10 points has singular values 6 and 2. What are the eigenvalues of its covariance matrix?
7. **Going back.** In problem 4 we kept only z. Rebuild the point from it. How far is the rebuilt point from the real $\mathbf{x}$ = `[2, 1]`?
8. **Units.** We run PCA on income (in dollars) and age (in years) without standardizing. What will PC 1 look like, and why?
9. **Reading a scree plot.** The eigenvalues are 4.0, 3.5, 0.3, 0.1 and 0.1. Where is the elbow? How many would you keep?
10. **Spot the leak.** A classmate fits PCA on the whole dataset, then splits it into training and test sets. What went wrong?

Answers

1. The average is 4. The squared distances are 4, 0 and 4, so the variance is $8/2 = 4$ and the standard deviation is 2.
2. $$C = \begin{bmatrix} 4 & -3 \\ -3 & 9 \end{bmatrix}$$
   The covariance is negative: when one goes up, the other tends to go down.
3. $(1 - \lambda)^2 = 0.5^2$, so $\lambda_1 = 1.5$ and $\lambda_2 = 0.5$. PC 1 holds $1.5/2 = 75\%$.
4. $0.6^2 + 0.8^2 = 1$, so yes. $z = 0.6(2) + 0.8(1) = 2$.
5. The total is 10. The first two keep $8/10 = 80\%$. Three keep 95%, so we need three.
6. $\lambda_1 = 36/9 = 4$ and $\lambda_2 = 4/9 \approx 0.44$.
7. The rebuilt point is $2 \times [0.6, 0.8] = [1.2, 1.6]$. It misses by $[0.8, -0.6]$, a distance of 1: the part of x along the direction we dropped.
8. Incomes vary by tens of thousands and ages by tens, so nearly all the variance is in income. PC 1 will point almost exactly along the income axis. Standardizing fixes it.
9. The elbow is after the second component. The first two hold $7.5/8 \approx 94\%$ of the variance, so keep two.
10. The axes were learned from the test rows too, so the test score is optimistic. Split first, fit on the training set only, then transform both.

Quick-check answers

p. 3: $5^6 = 15{,}625$ cells, and at most 20 of them have a point in them.

p. 7: Put `b = a` in `[8.2]` and you get `[8.1]`: a feature's covariance with itself is its variance.

p. 11: Every student is on the same side of average in both subjects, so every term of `[8.2]` is positive: the covariance is positive.

p. 18: Their standardized scores are `[0, 1]`, so `z = 0.71(0) + 0.71(1) = 0.71`.

p. 19: Before, every product was positive, which gave a covariance of 0.76. In the new axes the positive and negative products cancel, so the covariance is 0.